Conceptual Foundations of Statistical Learning

George Michailidis

gmichail@ucla.edu

STAT 426

Motivation - the General Regression Framework

- Goal: Predict a continuous outcome y based on input vector x.
- The Model:

$$y = f(x) + \epsilon$$

- Components:
  - $f(x)$: The unknown deterministic function that "links" the inputs x to the outcome variable y.
  - ϵ: Error with $\mathbb{E}(\epsilon) = 0, \text{Var}(\epsilon) = \sigma^2$.
- The Broad Strategy: We change how we parameterize $f(x)$ to capture different patterns in data.

1. Linear Regression (1900s)

Basics:

- Input: Feature vector $x \in \mathbb{R}^p$.
- Outcome: Continuous scalar $y \in \mathbb{R}$.
- Assumption: The relationship is globally linear and additive.

Parameterization of $f(x)$:

$$f(x) = \beta_0 + \sum_{j=1}^{p} \beta_j x_j = \beta^T x$$

- Pros: Highly interpretable, low variance.
- Cons: High bias if the true f is non-linear.

2. Polynomial Regression (1970-80s)

Basics:

- Extends linear regression by adding powers of features.
- Input: Scalar x (or vector).
- Assumption: $f(x)$ is smooth and global.

Parameterization of $f(x)$: (scalar version)

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d$$

- This is a Global Basis Expansion.

3. Spline Regression (1970-1990s)

Basics:

- Fits low-degree polynomials in separate regions defined by knots.
- Assumption: $f(x)$ is piecewise continuous with continuous derivatives.
- Locality: Changing data in one region only affects the fit locally.

Parameterization of $f(x)$ (Truncated Power Basis): (scalar input)

$$f(x) = \sum_{j=0}^{3} \beta_j x^j + \sum_{k=1}^{K} \theta_k (x - \xi_k)_+^3$$

Where ξ_k are the knots.

Visualizing Spline Bases

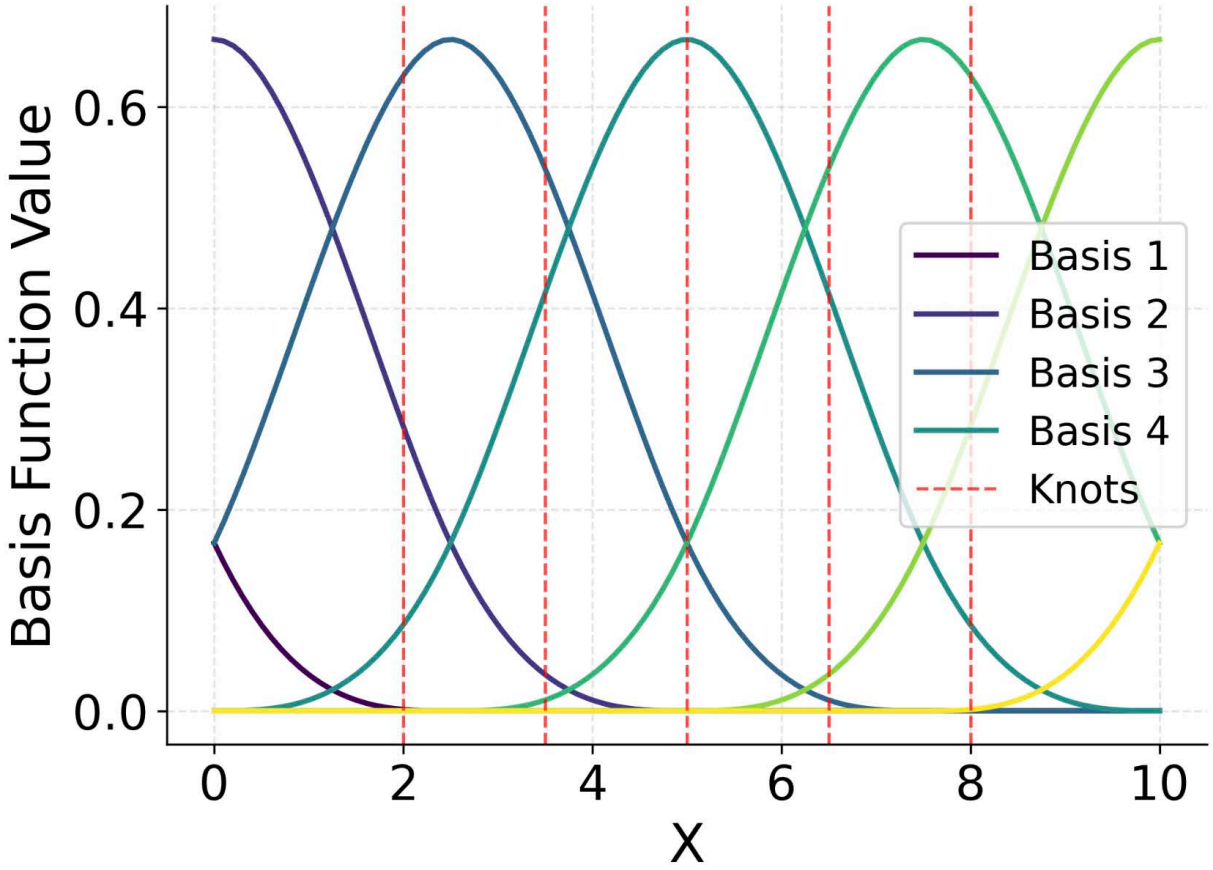

Figure 1: Bell-shaped curves overlapping locally. B-Splines allow for flexible local fitting without global oscillations.

Visualizing the Data Generation Mechanism

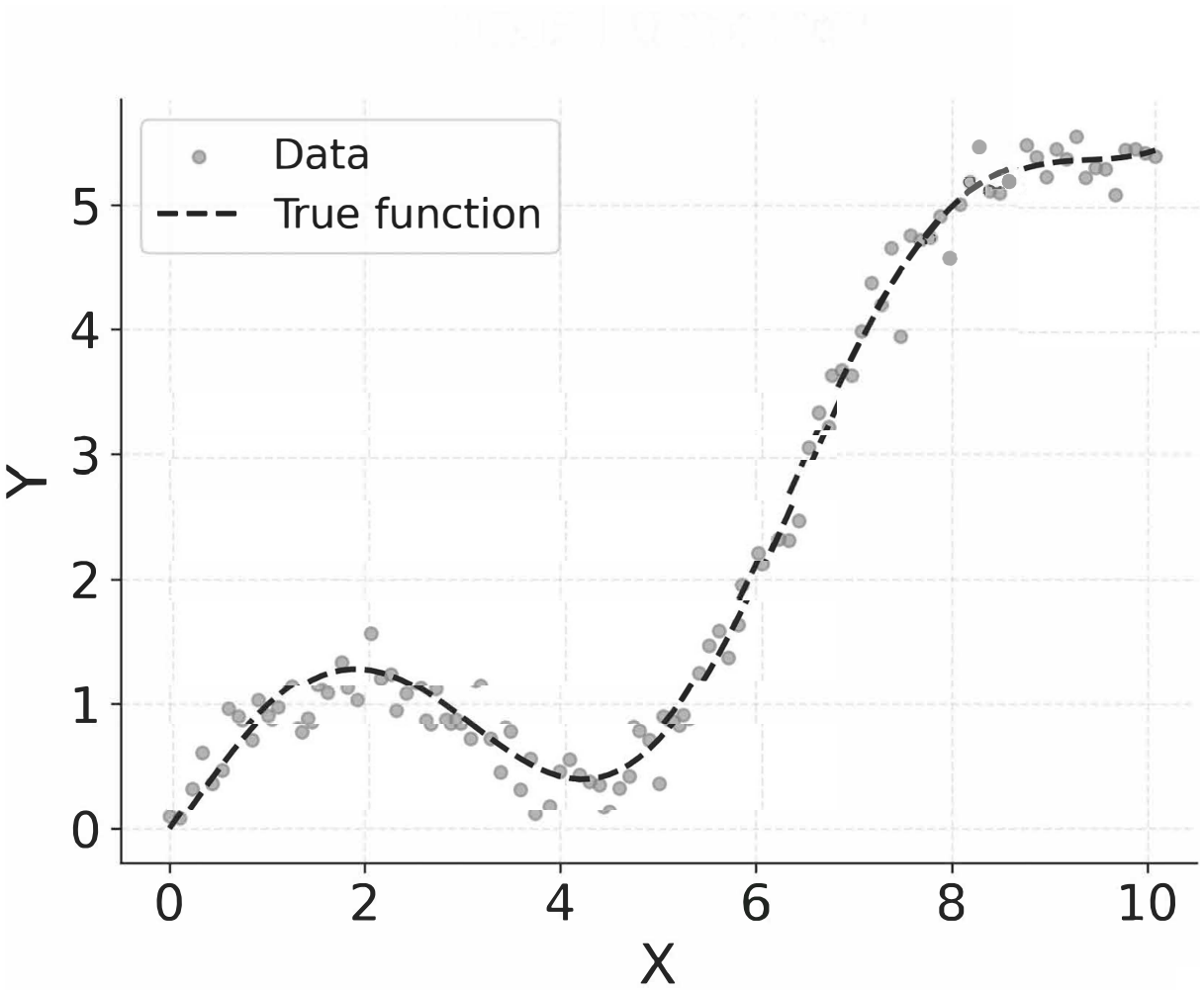

Figure 2: Data generating mechanism and data.

Visualizing Univariate Nonlinear Regressions

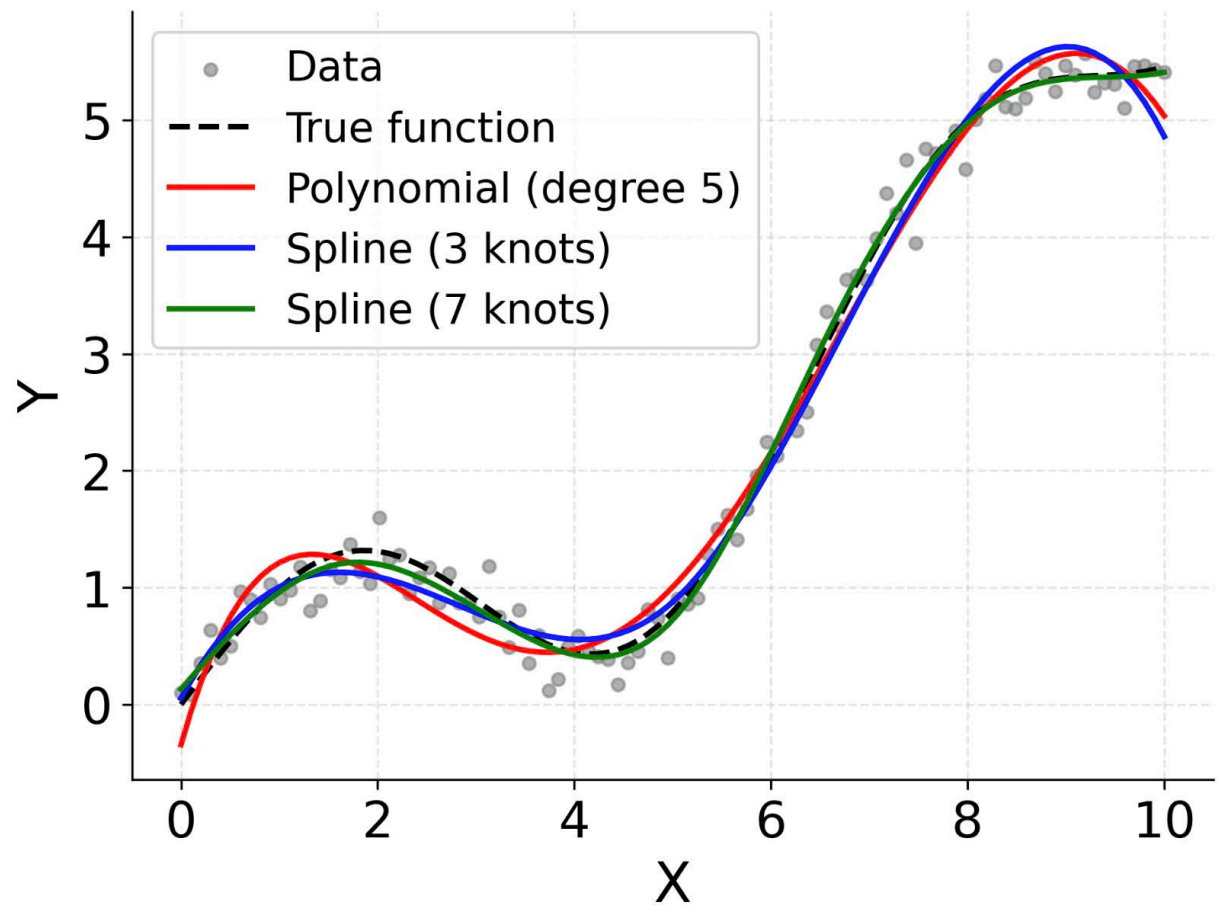

Figure 3: Data generating mechanism and data.

4. Fourier Regression (1960-70s)

Basics:

- Approximates function f as a sum of sines and cosines.
- Input: Typically time-series or periodic domain.
- Assumption: $f(x)$ is periodic or defined on a bounded interval.

Parameterization of $f(x)$: (scalar input)

$$f(x) = a_0 + \sum_{k=1}^{K} \left[a_k \cos\left(\frac{2\pi kx}{T}\right) + b_k \sin\left(\frac{2\pi kx}{T}\right) \right]$$

- Good for global frequency analysis, bad for local spikes.

Visualizing Fourier Bases

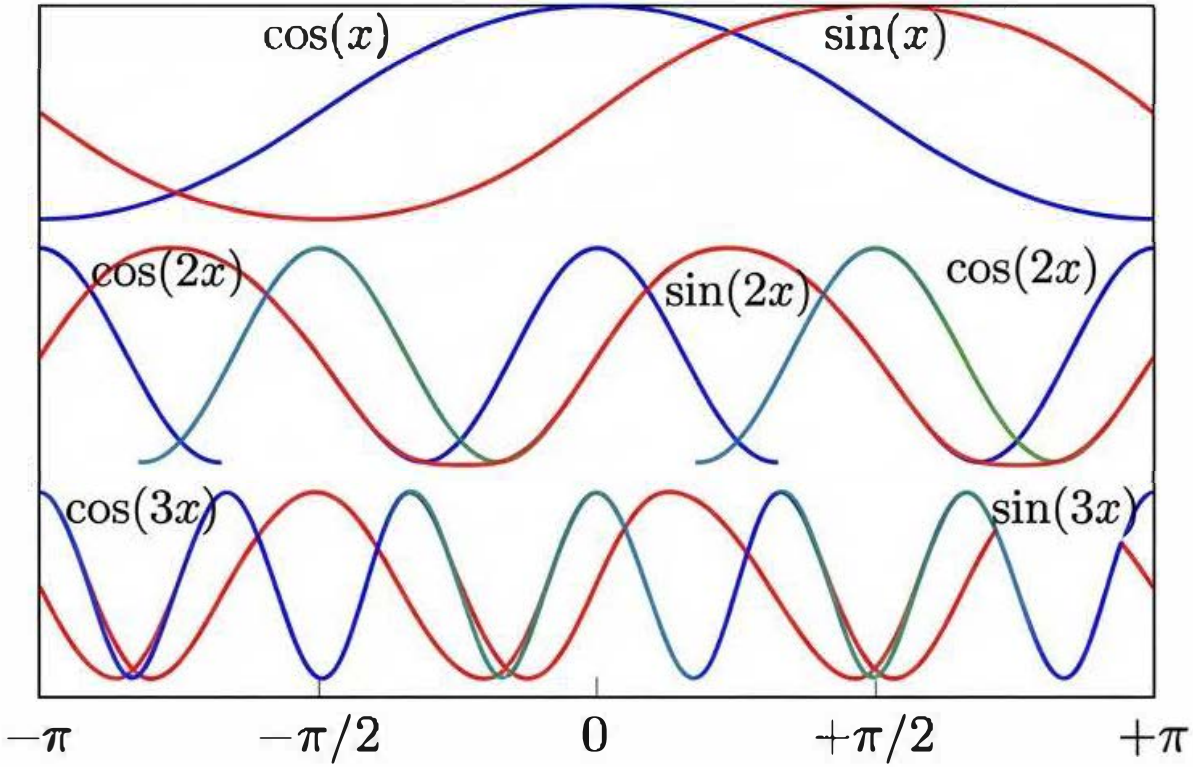

Figure 4: Fourier basis functions cover the entire domain (Global support).

5. Wavelet Regression (1990-2000s)

Basics:

- Uses basis functions localized in both time and frequency.
- Input: Signals, Images, non-stationary Time-series.
- Assumption: Function may contain discontinuities or sharp spikes.

Parameterization of $f(x)$: (scalar input)

$$f(x) = \sum_k c_{j_0,k}\phi_{j_0,k}(x) + \sum_{j=j_0}^{J}\sum_k d_{j,k}\psi_{j,k}(x)$$

- ϕ: Scaling function (Coarse approximation).
- ψ: Mother wavelet (Detail coefficients).

Visualizing Wavelet Bases

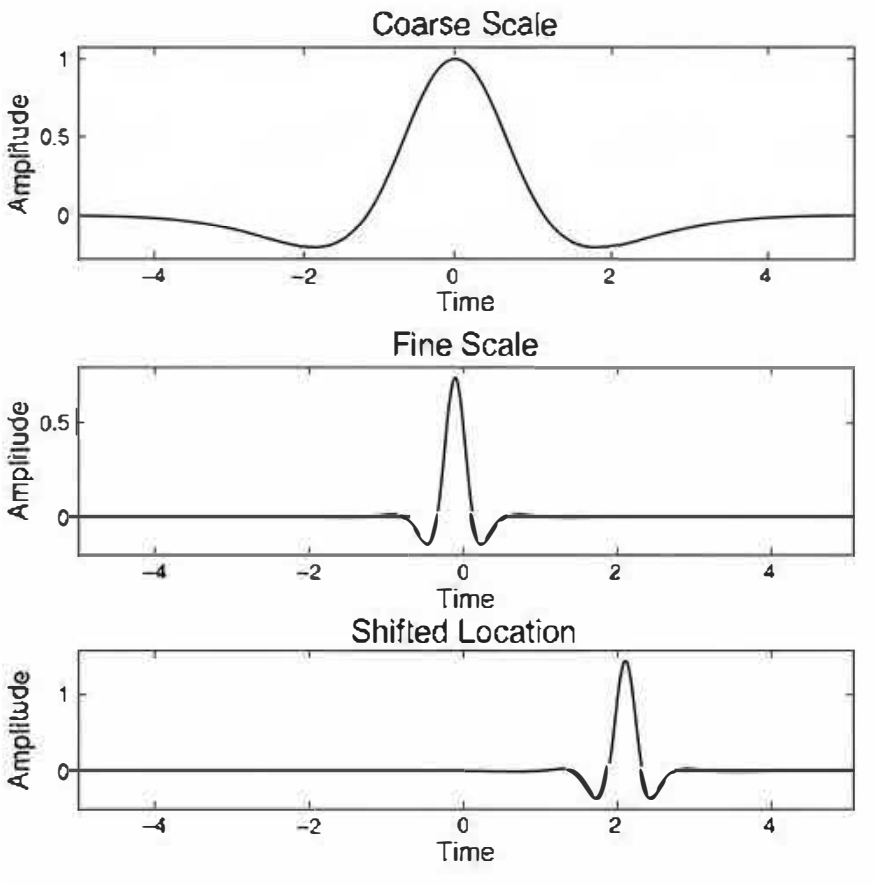

Figure 5: Wavelets adapt to features at different scales and locations.

6. Kernel Regression (The Kernel Trick) (1990-2000s)

The Concept:

- Instead of manually creating basis functions, we map inputs implicitly to an infinite-dimensional feature space $\mathcal{H}$.
- Goal: Perform Linear Regression in this high-dimensional space.

The "Trick": We avoid computing the mapping $\phi(x)$ explicitly. We only need the dot products, which are defined by the Kernel Function:

$$\langle \phi(x), \phi(x') \rangle = K(x, x')$$

Parameterization (Dual Representation): According to the Representer Theorem, the optimal function is a linear combination of kernels centered at training points:

$$f(x) = \sum_{i=1}^{N} \alpha_i K(x, x_i)$$

- Parameters: α_i (Dual coefficients), solved via $(K + \lambda I)\alpha = y$.
- Advantage: Captures complex non-linearities without defining specific basis features.

Transition to Classification

- Change in Outcome: y is categorical (discrete).
- Goal: Model the Probability of class membership.

The Link Function: Instead of predicting y directly, we map $f(x)$ to $[0, 1]$.

$$P(y = k|x) = g(f(x))$$

- $f(x)$ remains one of the regression models discussed (Linear, Kernel, etc.).
- $g(\cdot)$ is the activation (e.g., Sigmoid, Softmax).

Binary Classification (Logistic)

Basics:

- Outcome: $y \in \{0, 1\}$.
- Assumption: Log-odds is linear (or basis expanded).

Parameterization:

$$p(x) = \frac{1}{1 + e^{-f(x)}} = \sigma(\beta^T x)$$

- Decision Boundary: Defined where $f(x) = 0$.
- If we use a Kernel for $f(x)$, we get Kernel Logistic Regression or SVM behavior.

Kernel Classification (SVM Context)

Concept: If classes are not linearly separable, project to high dimensions using $K(x, x')$.

Parameterization of $f(x)$:

$$f(x) = \sum_{i=1}^{N} \alpha_i y_i K(x, x_i) + b$$

- Outcome: $\hat{y} = \text{sign}(f(x))$.
- Only support vectors (where $\alpha_i \neq 0$) matter.

Multi-Class Classification

Basics:

- Outcome: $y \in \{1, 2, \ldots, K\}$.
- Assumption: Classes are mutually exclusive.

Parameterization (Softmax / Multinomial): We learn K functions $f_1(x), \ldots, f_K(x)$.

$$P(y = k|x) = \frac{e^{f_k(x)}}{\sum_{j=1}^{K} e^{f_j(x)}}$$

- Select class k with the highest probability.
- Often $f_k(x) = \beta_k^T x$ (Linear boundaries between classes), but other parameterization can also be used.

7. Neural Networks (MLP Regression)
(basic formulation going back to 1960s, and exploding in popularity since 2010)

Basics:

- Concept: A "Universal Approximator" composed of layers of linear combinations and non-linear activations.
- Input: $x \in \mathbb{R}^p$. Outcome: $y \in \mathbb{R}$.

Parameterization (Single Hidden Layer):

$$f(x) = \beta_0 + \sum_{m=1}^{M} \beta_m \sigma(\alpha_m^T x + b_m)$$

- $\sigma(\cdot)$: Non-linear activation function (e.g., ReLU, Tanh).
- The term $z_m = \sigma(\alpha_m^T x + b_m)$ acts as a Derived Feature.

Key Distinction (Adaptive vs. Fixed):

- Splines/Fourier/Wavelets: The basis functions are fixed beforehand (feature engineering).
- **MLP:** The basis functions are learned. The parameters α inside the activation are optimized alongside β.

A Unified Perspective

We have looked at a diverse collection of models:

- Linear & Polynomial: Global, simple parametric forms.
- Splines & Wavelets: Local, piecewise forms.
- Fourier: Spectral/Frequency domain forms.
- Kernels: Similarity-based, infinite-dimensional forms.
- Feed-forward Neural Networks: Compositional, adaptive forms.

The Key Question:

Is there a single technical framework that unifies all these different approaches?

The Answer: Statistical Learning Theory

Yes. They are all instances of Supervised Learning.

Regardless of the specific parameterization (basis functions, kernels, neurons), every model presented adheres to the same fundamental rules of Statistical Learning Theory.

The Common Theme:

1. Data Generation Mechanism: We assume data come from an unknown distribution $\mathbb{P}(X, Y)$.
2. Evaluation: We measure "success" using a Loss Function (e.g., Squared Error, Cross-Entropy).
3. Objective: We try to minimize the expected error (Risk) using the observed data (Empirical Risk Minimization).

Next, we will formalize this framework and see how it guides us in choosing the right model and avoiding overfitting $\Longrightarrow$ "good generalization".

Setting the Stage: The Statistical Setup

The Environment:

- We assume a true, but unknown joint distribution $\mathbb{P}(X, Y)$.
- Inputs: $X \in \mathcal{X}$ (e.g., images, vectors).
- Labels: $Y \in \mathcal{Y}$ (e.g., classes, real numbers).

The Data:

- We observe a training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$.
- Assumption: Data points are drawn **i.i.d.** (independently and identically distributed) from $\mathbb{P}(X, Y)$ (generalizes to specific forms of dependence, like time series data).

The Goal:

- Find a function $f : \mathcal{X} \to \mathcal{Y}$ that predicts Y given X.
- We want $f(x)$ to work well on unseen data (Generalization).

Hypothesis Classes

To find the best function f, we must first restrict our search to a specific set of candidate functions.

Definition: Hypothesis Class $\mathcal{H}$

A set of functions we are willing to consider.

$$\mathcal{H} = \{f_\theta : \theta \in \Theta\}$$

Connecting to Previous Models:

- **Linear Regression:** $\mathcal{H}_{lin} = \{f(x) = \beta^T x \mid \beta \in \mathbb{R}^p\}$
- **Splines:** $\mathcal{H}_{spline} = \{\text{Piecewise polynomials with knots } \xi\}$
- **Neural Networks:** $\mathcal{H}_{MLP} = \{\text{Compositions of linear + non-linear maps}\}$

Key Concept: Selecting the model is equivalent to selecting the Hypothesis Class $\mathcal{H}$.

Inductive Bias

- Without assumptions, learning is impossible (No Free Lunch Theorem).
- Inductive Bias: The set of assumptions we make about the relationship between X and Y by choosing a specific $\mathcal{H}$.

Examples of Inductive Bias:

- **Linear:** We assume gradients are constant globally.
- **Kernel/RBF:** We assume y values are similar if x values are close (smoothness).
- **CNNs (Images):** We assume spatial invariance.

Quantifying Success: The Loss Function

How do we know if a specific $f \in \mathcal{H}$ is "good"?

Definition: Loss Function

A function $L : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_+$ that measures the cost of predicting $\hat{y} = f(x)$ when the true label is y.

$$L(y, f(x))$$

Properties:

- $L(y, y) = 0$ (No cost for perfect prediction).
- Penalizes deviations from the truth.
- Choice of L depends on the task (Regression vs. Classification).

Regression Losses (Continuous Outcome)

If $Y \in \mathbb{R}$, common choices include:

1. Squared Error (ℓ_2 Loss):

$$L(y, f(x)) = (y - f(x))^2$$

- Pros: Sensitive to outliers, mathematically convenient (differentiable everywhere), leads to mean estimation.
- Used in: OLS, Splines, Kernel Ridge.

2. Absolute Error (ℓ_1 Loss):

$$L(y, f(x)) = |y - f(x)|$$

- Pros: Robust to outliers, leads to median estimation.
- Cons: Not differentiable at 0.

Visualizing Regression Losses

- Huber Loss: A hybrid approach.

$$L_\delta(a) = \begin{cases} \frac{1}{2}a^2 & \text{for } |a| \leq \delta, \\ \delta(|a| - \frac{1}{2}\delta), & \text{otherwise.} \end{cases}$$

- Behaves like ℓ_2 near zero (differentiable) and ℓ_1 far away (robust).

Classification Losses (Discrete Outcome)

If $Y \in \{-1, +1\}$, the ultimate goal is to minimize errors.

1. The 0/1 Loss (Gold Standard):

$$L_{0/1}(y, f(x)) = \mathbb{I}(y \neq \text{sign}(f(x)))$$

- Problem: Non-convex, non-differentiable, combinatorial optimization (NP-hard).
- Solution: We use Convex Surrogates.

Surrogate Loss Functions

We optimize an upper bound or approximation of the 0/1 loss.

- Hinge Loss (SVM):

$$L(y, f(x)) = \max(0, 1 - yf(x))$$

  Promotes the concept of a *margin*.

- Logistic Loss (Logistic Reg):

$$L(y, f(x)) = \log(1 + \exp(-yf(x)))$$

  Probabilistic interpretation.

- Exponential Loss (Boosting):

$$L(y, f(x)) = \exp(-yf(x))$$

Visualizing Classification Losses

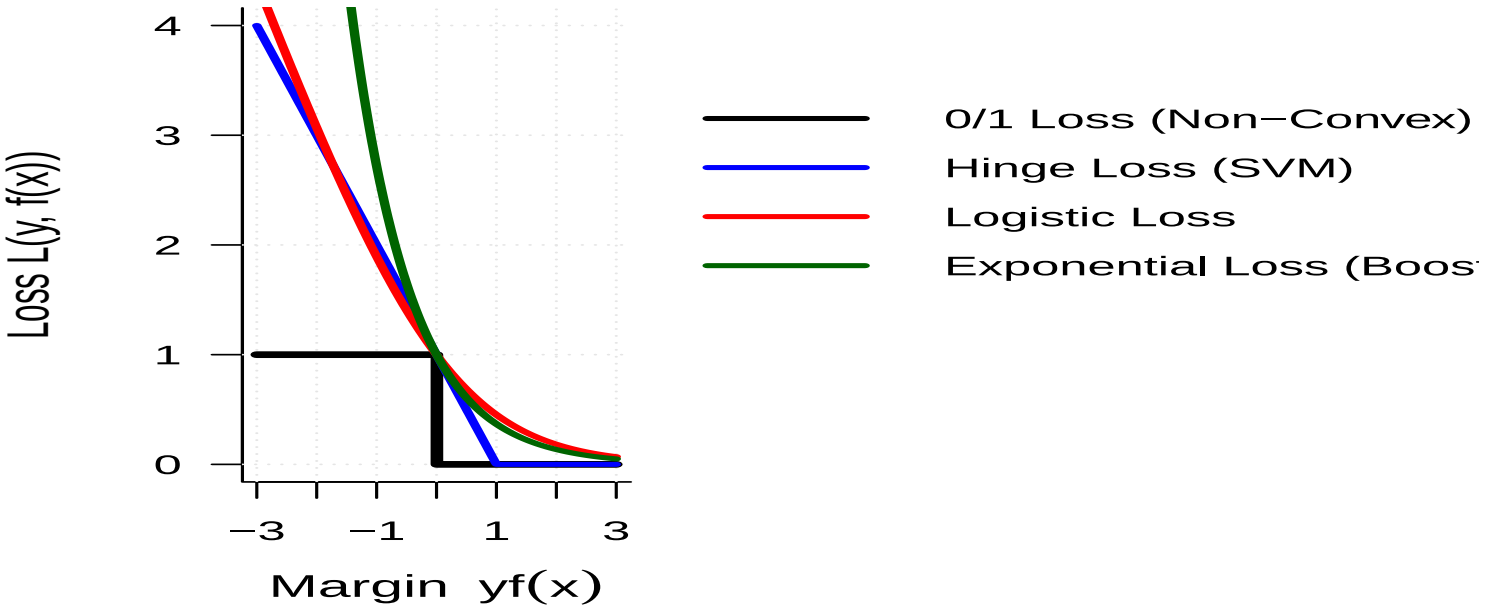

Figure 6: Comparison of 0/1 Loss vs Convex Surrogates (Hinge, Logistic, Exponential). Note that surrogates are differentiable and also convex functions.

Key Takeaway: We optimize the surrogate loss to find parameters β, aiming to approximate fairly accurately the ideal 0/1 loss function.

True Risk (The Ideal Objective)

We want to perform well on all possible data drawn from $\mathbb{P}(X, Y)$.

Definition: True Risk (Generalization Error)

The expected loss over the data distribution:

$$R(f) = \mathbb{E}_{X,Y}[L(Y, f(X))] = \int L(y, f(x))dP(x, y)$$

The Fundamental Problem:

- We do not know $\mathbb{P}(X, Y)$.
- Therefore, we can not compute $R(f)$ or minimize it directly.

Empirical Risk (The Proxy)

Since we can not compute the expectation, we approximate it using the sample mean over our training data $\mathcal{D}$.

Definition: Empirical Risk

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^{n} L(y_i, f(x_i))$$

Assumptions:

- Data are i.i.d.: $\mathcal{D} = (x_i, y_i) \overset{\text{i.i.d.}}{\sim} \mathbb{P}$.
- Loss is bounded: $L(\cdot, \cdot) \in [0, 1]$ (or at least has finite range/moments).

, Then, by the Law of Large Numbers (LLN), $\hat{R}(f) \to R(f)$ as $n \to \infty$ for a **fixed** f.

Empirical Risk Minimization (ERM)

This leads to the dominant paradigm in machine learning:

The ERM Principle

Choose the function $\hat{f}$ from the hypothesis class $\mathcal{H}$ that minimizes the empirical risk (training error):

$$\hat{f} = \underset{f \in \mathcal{H}}{\operatorname{argmin}}\, \hat{R}(f) = \underset{f \in \mathcal{H}}{\operatorname{argmin}}\, \frac{1}{n} \sum_{i=1}^{n} L(y_i, f(x_i))$$

Key subtlety: $\hat{f}$ depends on the same data used to compute $\hat{f}$.

This data-dependence is the source of overfitting risk: $\hat{f}$ might exploit random sample fluctuations.

ERM Examples

Most classical ML algorithms are just ERM with different Loss Functions (L) and Hypothesis Classes ($\mathcal{H}$).

1. Linear Regression:
   - $\mathcal{H}$: Linear functions $f(x) = \beta^T x$
   - L: Squared loss $(y - \hat{y})^2$
   - Optimization: $\text{argmin}_\beta \sum (y_i - \beta^T x_i)^2$
2. Support Vector Machines (SVM):
   - $\mathcal{H}$: Linear separators (hyperplanes)
   - L: Hinge loss $\max(0, 1 - y_i f(x_i))$
   - Optimization: $\text{argmin}_{w,b} \sum \max(0, 1 - y_i(w^T x_i + b))$

The Failure of Naive ERM

If $\mathcal{H}$ is too complex (e.g., high-degree polynomials, unregularized neural nets), ERM can fail.

Why?

- We can find an f that effectively "memorizes" the training data.
- $\hat{R}(f) \approx 0$ (Training error is very small, possibly zero).
- $R(f)$ is very large (Test error is high).

This is Overfitting. The empirical risk is no longer a good estimate of the true risk.

The Theoretical Limit: Bayes Optimal Predictor - I

To understand why models fail, we compare them to the theoretical best:

Definition: Bayes Optimal Predictor (f^*)

The function that achieves the minimal possible risk over *all* functions (not restricted to $\mathcal{H}$).

$$f^*(x) = \underset{\text{all functions } f}{\operatorname{argmin}} \; R(f)$$

Note: obtaining f^* is impossible in practice, since it requires knowledge of the true distribution $\mathbb{P}(X, Y)$.

The Theoretical Benchmark: Bayes Optimal Predictor - II

What does f^* look like theoretically?

- Regression (ℓ_2 Loss):
  The optimal predictor is the Conditional Mean.

$$f^*(x) = \mathbb{E}[Y \mid X = x]$$

- Classification (0/1 Loss):
  The optimal predictor is the Bayes Classifier (most probable class).

$$f^*(x) = \operatorname*{argmax}_{k \in \mathcal{Y}} P(Y = k \mid X = x)$$

Irreducible Error (Bayes Risk)

Even the optimal f^* is not perfect. Since Y is rarely a deterministic function of X (due to noise or missing features), the risk $R(f^*) > 0$. This lower bound is called the **Irreducible Error**.

Decomposing Excess Risk: the Fundamental Bias-Variance Trade-off

We can decompose the Excess Risk of our learned model $\hat{f}$ in class $\mathcal{H}$ into two distinct sources of error:

Let $f^*_{\mathcal{H}} = \text{argmin}_{f \in \mathcal{H}} R(f)$ be the best possible function *within* our hypothesis (model) class $\mathcal{H}$.

$$\underbrace{R(\hat{f}) - R(f^*)}_{\text{Total Excess Risk}} = \underbrace{(R(f^*_{\mathcal{H}}) - R(f^*))}_{\text{Approximation Error}} + \underbrace{(R(\hat{f}) - R(f^*_{\mathcal{H}}))}_{\text{Estimation Error}}$$

- **Approximation Error (Bias):**
  The penalty for restricting our search to class $\mathcal{H}$.
  - *Intuition:* Does the model family even contain the truth? (e.g., trying to fit a sine wave with a linear model).
- **Estimation Error (Variance):**
  The penalty for having finite training data n.
  - *Intuition:* We selected $\hat{f}$ based on noisy data. How much worse is $\hat{f}$ (the one we found) compared to $f^*_{\mathcal{H}}$ (the best one we *could* have found)?

Visualizing the Bias-Variance Trade-off

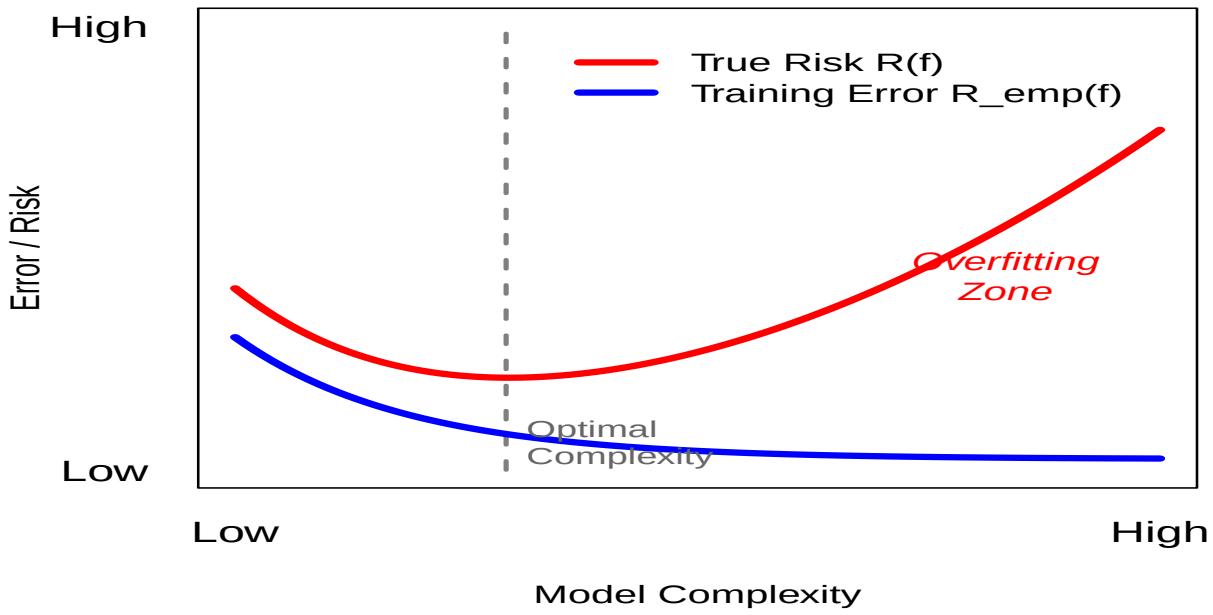

Figure 7: As model complexity increases, Training Error ($\hat{R}$) drops, but True Risk (R) eventually rises.

- Underfitting (Left): High bias, model is too simple to capture the signal.
- Overfitting (Right): High variance, model memorizes noise in $\mathcal{D}$.

Examples of Approximation vs. Estimation

Small $\mathcal{H}$ (e.g., Linear Regression):

- High Approximation Error (High Bias).
- Low Estimation Error (Low Variance).
- Risk: Underfitting.

Large $\mathcal{H}$ (e.g., High-deg Polynomial, Deep MLP):

- Low Approximation Error (Low Bias).
- High Estimation Error (High Variance).
- Risk: Overfitting.

What can we say about Excess Risk?

The Challenge: Note that the Excess Risk depends on the true data distribution $\mathbb{P}(X, Y)$, which is **unknown**!

The Bottleneck: The difficulty lies in the Estimation Error term.

- We need to know the true risk $R(\hat{f})$.
- We only have access to the empirical risk $\hat{R}(\hat{f})$.

The Implication: Unless we can guarantee that our proxy is accurate (i.e., $\hat{R}(\hat{f}) \approx R(\hat{f})$), we do not have control over the Excess Risk.

How to proceed? We need to bound the difference between these two quantities using probability theory (Hoeffding + Union Bound).

The Generalization Gap

While the previous slide decomposed true performance (Excess Risk), in practice we only observe the Training Error.

Definition: Generalization Gap

The difference between the True Risk (test error based on a test data set) and the Empirical Risk (training error based on $\mathcal{D}$) of our learned model $\hat{f}$:

$$\text{GenGap} = R(\hat{f}) - \hat{R}(\hat{f})$$

Why does this gap exist?

- Optimization Bias: We chose $\hat{f}$ specifically because it minimized $\hat{R}$.
- Overfitting: The model may have adapted to the specific random noise of the training data $\mathcal{D}$.
- Result: $\hat{R}(\hat{f})$ is an optimistically biased estimate. It is almost always smaller than the true risk $R(\hat{f})$.

Digression: A useful tool

Hoeffding's inequality provides a high-probability bound for a single, fixed function f:

$$\mathbb{P}\big(|\hat{R}(f) - R(f)| > \epsilon\big) \leq 2\exp(-2n\epsilon^2).$$

So as sample size n increases, the difference vanishes exponentially fast

But ERM picks $\hat{f}$ by searching over $\mathcal{H}$.

So we need a simultaneous (uniform) control over all $f \in \mathcal{H}$.

The Generalization Gap and the Bridge

Define the uniform/generalization gap:

$$\Delta_n(\mathcal{H}) \;=\; \sup_{f \in \mathcal{H}} |\hat{R}(f) - R(f)|.$$

Key inequality (bridge): If $\Delta_n(\mathcal{H}) \leq \epsilon$ then

$$R(\hat{f}) - R(f^*_{\mathcal{H}}) \leq 2\epsilon.$$

Proof sketch:

$$\begin{aligned} R(\hat{f}) - R(f^*_{\mathcal{H}}) &= \left[R(\hat{f}) - \hat{R}(\hat{f})\right] + \left[\hat{R}(\hat{f}) - \hat{R}(f^*_{\mathcal{H}})\right] + \left[\hat{R}(f^*_{\mathcal{H}}) - R(f^*_{\mathcal{H}})\right] \\ &\leq \epsilon + 0 + \epsilon \quad (\text{since } \Delta_n \leq \epsilon,\ \hat{R}(\hat{f}) \leq \hat{R}(f^*_{\mathcal{H}})) \\ &= 2\epsilon. \end{aligned}$$

So uniform control of the generalization gap directly bounds the estimation error.

Back to the Problem of Many Hypotheses

But wait! In ERM, we are not testing one fixed function. We search through the entire class $\mathcal{H}$ to pick the "best" one.

The "Lucky Coin" Analogy:

- If **1 person** flips a fair coin 10 times, getting 10 heads is rare ($P \approx 0.001$).
- If **1,000 people** flip coins, it is **likely** that *someone* gets 10 heads by pure luck.
- *Implication:* If $\mathcal{H}$ is large, we might find a "lucky" model that fits the noise perfectly.

Digression: Another useful tool

Union-bound idea: To control the learned predictor $\hat{f}$ we need bounds that hold *simultaneously* for all $f \in \mathcal{H}$ (uniform control).

$$\mathbb{P}\big(\Delta_n(\mathcal{H}) > \epsilon\big) \;=\; \mathbb{P}\Big(\exists f \in \mathcal{H} : \; |R(f) - \hat{R}(f)| > \epsilon\Big) \;\leq\; \sum_{f \in \mathcal{H}} \mathbb{P}\big(|R(f) - \hat{R}(f)| > \epsilon\big).$$

(This is the union bound; the per-function probabilities are supplied by Hoeffding.)

The Generalization Bound (Finite Class)

Combining Hoeffding (for one f) and the Union Bound (summing over $|\mathcal{H}|$ functions), we obtain the fundamental guarantee:

Uniform Convergence Bound

$$\mathbb{P}(\Delta_n(\mathcal{H}) > \epsilon) \leq 2|\mathcal{H}| \exp(-2n\epsilon^2)$$

Let's parse this term by term:

1. n (Sample Size): In the exponent with a negative sign. As n grows, the probability of overfitting vanishes exponentially fast.
2. $|\mathcal{H}|$ (Model Complexity): The size of the hypothesis class acts as a multiplier.
   - As we consider more models (higher complexity), $|\mathcal{H}|$ grows.
   - We need more data (n) to counteract the $\ln(|\mathcal{H}|)$ factor in the exponent.

The Fundamental Trade-off

To maintain a low Estimation Error, if you increase complexity ($|\mathcal{H}| \uparrow$), you **must** increase data ($n \uparrow$).

Where Does Approximation Error Come In?

Recall the excess-risk decomposition:

$$R(\hat{f}) - R(f^*) = \underbrace{R(f^*_{\mathcal{H}}) - R(f^*)}_{\text{Approximation error}} + \underbrace{R(\hat{f}) - R(f^*_{\mathcal{H}})}_{\text{Estimation error}}.$$

What we controlled:

- The analysis bounds only the estimation error:

$$R(\hat{f}) \leq R(f^*_{\mathcal{H}}) + (\text{estimation bound}).$$

What we did not control:

- Approximation error depends on how expressive $\mathcal{H}$ is.
- It is not reduced by more data and can not be bounded by Hoeffding-Union bound type arguments.

Key message

Generalization theory controls *learning from data*, not *modeling error*.

How Can Approximation Error Be Reduced?

Recall: Approximation error measures how well the true target can be represented within the hypothesis class: Approximation Error $= R(f^*_{\mathcal{H}}) - R(f^*)$.

Key idea: Make the model class more expressive.

- Replace a small class $\mathcal{H}$ with a larger one.
- A richer class can represent more complex functions and reduce bias.

Typical strategies:

- Increase polynomial degree.
- Add more basis functions (Fourier, wavelets, splines).
- Increase neural network width or depth.

Limiting case:

- Consider a sequence of classes

$$\mathcal{H}_1 \subset \mathcal{H}_2 \subset \cdots$$

with

$$\lim_{k \to \infty} R(f^*_{\mathcal{H}_k}) = R(f^*).$$

- In principle, an infinite class can make approximation error arbitrarily small.

But beware

Enlarging $\mathcal{H}$ increases estimation error and can break uniform convergence unless additional structure or regularization is imposed.

Controlling Estimation Error in Practice: Structural Risk Minimization (Regularization)

To fix overfitting, we modify the ERM objective. We penalize complexity.

Regularized Objective

$$\hat{f} = \underset{f \in \mathcal{H}}{\operatorname{argmin}}\, \hat{R}(f) + \lambda \Omega(f)$$

- $\hat{R}(f)$: Fit the data (minimize bias).
- $\Omega(f)$: Keep model simple (minimize variance).
- λ: Hyperparameter controlling the trade-off.

Parametric Regularization (Ridge & Lasso)

Recall Linear Regression: $f(x) = \beta^T x$.

Ridge Regression (ℓ_2 Penalty):

$$\Omega(f) = \|\beta\|_2^2 = \sum \beta_j^2$$

- Shrinks coefficients toward zero.

Lasso Regression (ℓ_1 Penalty):

$$\Omega(f) = \|\beta\|_1 = \sum |\beta_j|$$

- Induces Sparsity (feature selection).

Visualizing ℓ_1 vs ℓ_2 Regularization

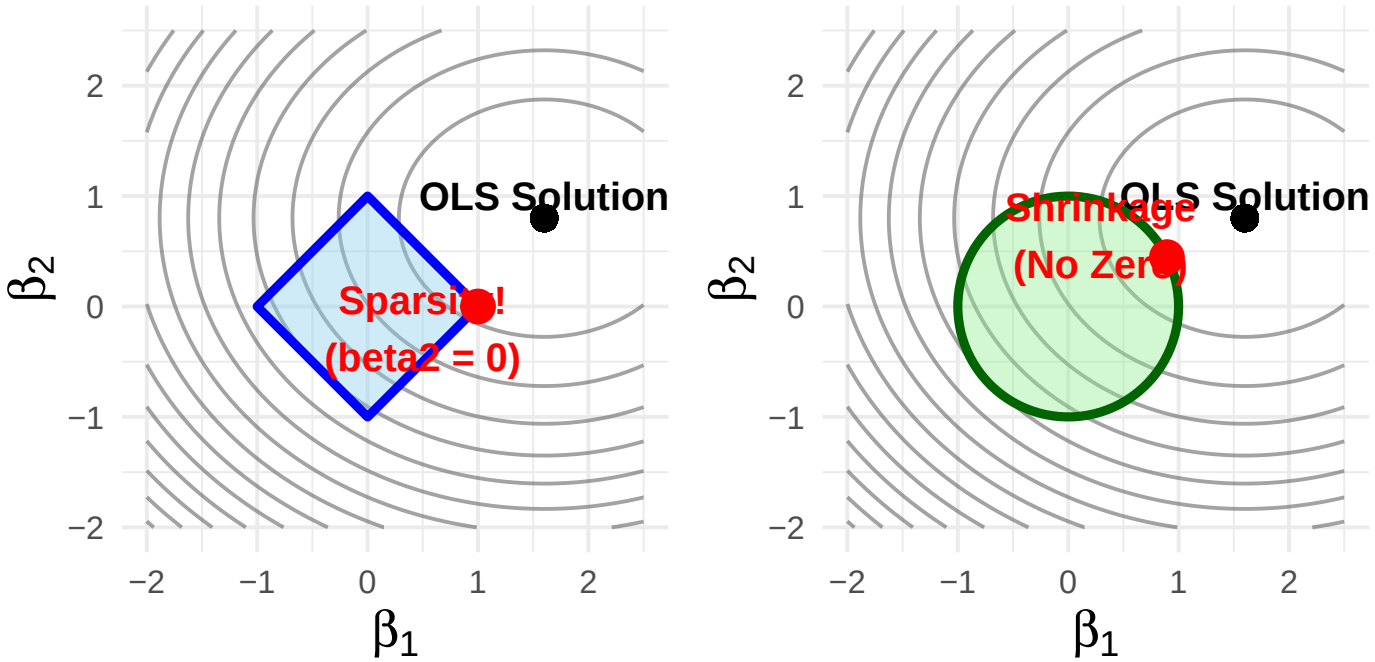

Figure 8: ℓ_1 and ℓ_2 regularizers in regression.

Function Space Regularization (Splines/Kernels)

Sometimes we regularize the function properties directly (Sobolev Norm).

In spline regression the objective becomes to minimize:

$$\sum_{i=1}^{n}(y_i - f(x_i))^2 + \lambda \int [f''(x)]^2 dx$$

- $\int [f''(x)]^2 dx$: Measures the "wiggliness" or curvature.
- **High λ:** Forces f to be linear (0 curvature).
- **Low λ:** Allows wiggly interpolation.

The "Deep Learning" Paradox

Recall the Bound:

$$P(\text{Excess Risk} > \epsilon) \leq 2|\mathcal{H}| \exp(-2n\epsilon^2)$$

The Conflict:

- For an MLP (and many other deep learning architectures), the hypothesis class $|\mathcal{H}|$ is determined by the number of parameters.
- Modern networks have **millions** of parameters (p), often where $p \gg n$ (more parameters than data points).
- Implication: The bound becomes vacuous (∞). Classical theory predicts Deep Nets should simply memorize noise and fail to generalize.

The Reality:

- Deep Networks generalize surprisingly well.
- Why? We need another way to measure "Complexity" of the class $\mathcal{H}$.

Resolving the Paradox: Effective Capacity

1. It's not about the *Count*, it's about the *Norm*.
   - Modern theory (e.g., Rademacher Complexity) suggests complexity depends on the magnitude of weights in the MLP ($||W||_F$), not just the number of weights.
   - Intuitively, a network with 1M parameters, but very small weights behaves like a simpler linear model.
2. Implicit Regularization (Double Descent):
   - As we increase complexity beyond the interpolation point (zero training error), test error often decreases again.
   - Why? The optimization algorithm (Stochastic Gradient Descent) to train an MLP (or other deep neural nets) prefers "simple" (minimum norm) solutions among the infinite zero-error solutions.

Connecting Regularization to Theoretical Insights for Deep NN

To make the generalization bound work, we use regularization to reduce the "Effective Complexity" of $\mathcal{H}$.

1. Weight Decay (ℓ_2 Reg) $\rightarrow$ Norm-Based Capacity
   - Mechanism: Minimize $||W||^2$.
   - Theory: Restricts search to a small ball in parameter space. The *accessible* $\mathcal{H}$ is much smaller than the full space.
2. Dropout $\rightarrow$ Ensemble Averaging
   - Mechanism: Randomly drop neurons during training.
   - Theory: Approximates an ensemble of 2^n thinned networks, reducing Variance (Estimation Error).
3. Early Stopping $\rightarrow$ Algorithmic Regularization
   - Mechanism: Stop when validation loss (to be discussed later) rises.
   - Theory: The optimizer starts at zero (simple). Stopping early prevents it from reaching the complex, "wiggly" regions of $\mathcal{H}$ that overfit.

Solving the ERM Problem

How do we actually find $\hat{f}$?

- **Analytical Solution:** Possible for OLS, Ridge, Kernel Ridge.

$$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$$

- **Iterative Optimization:** Required for Lasso, Logistic, Neural Nets.

Gradient Descent

Algorithm 1 Gradient Descent (GD)

```
Initialize parameters θ (e.g., random)
while not converged do
    Compute gradient of loss: g = ∇_θ R̂(f_θ)
    Update: θ ← θ − η · g
end while
```

- η: Learning rate.
- Issue: Computing ∇ over all N points is slow for big data.

Stochastic Gradient Descent (SGD)

Approximate the gradient using a single point (or mini-batch).

$$\theta \leftarrow \theta - \eta \nabla_\theta L(y_i, f_\theta(x_i))$$

- Pros: Faster updates, better scaling, noise helps escape shallow local minima (in non-convex landscapes like NNs).
- Cons: Noisy convergence.
- Fix: adaptive momentum methods (e.g., ADAM and variants)

Convexity vs Non-Convexity

Convex Problems:

- Linear/Ridge/Lasso/SVM.
- One global minimum.
- Convergence guaranteed.

Non-Convex Problems:

- Neural Networks.
- Many local minima.
- Initialization matters significantly.

Summary: The Big Picture

1. The Goal: Minimize True Risk $R(f)$.
2. The Reality: We only have Data $\mathcal{D}$, so we minimize Empirical Risk $\hat{R}(f)$.
3. The Tools (Hypothesis Classes):
   - Linear/Spline/Kernel (Interpretable, specific assumptions).
   - Neural Networks (Universal approximators, data hungry).
4. The Safety Valve (Regularization): Prevents Overfitting by penalizing complexity (ℓ_1, ℓ_2).
5. The Engine (Optimization): Gradient Descent / SGD.